Global Optimization

CS 1622

Jarrett Billingsley

Class Announcements

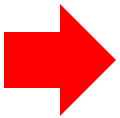
- **exam 1 grades** were posted on Monday, if you missed that
  - if you have questions about it, please ask me in private
- don't forget project 4 is due on Saturday (or late Sunday)!

From Local to Global

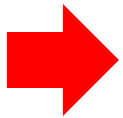
Local optimization only gets you so far

- last time we saw that optimizations like **copy propagation** and **dead store elimination** could greatly simplify our code.

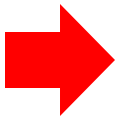
```
let x = 5 * 5;
return x;
```



```
x = 5 * 5
$t0 = x
return
```



```
x = 25
$t0 = x
return
```



```
$t0 = 25
return
```

but local optimizations only work on a **single basic block,** which means they have two significant weaknesses:

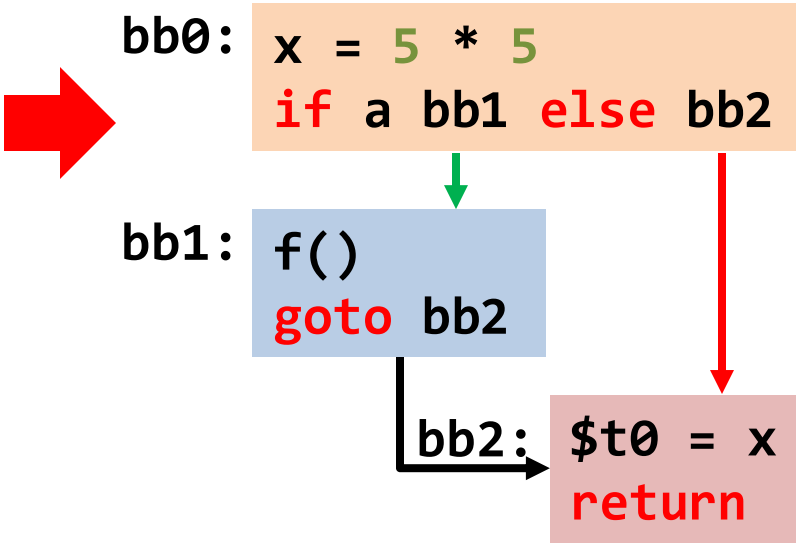
1. real functions can have **many** BBs, and they **connect to *and affect* each other** in nontrivial ways.

2. many of those BBs are **short,** only a few instructions long, meaning there just isn't much opportunity for optimization.

Stumbling basic block

- if we make that code just a little more complex... (assume **a** is an argument)

```
let x = 5 * 5;
if a { f(); }
return x;
```



we can still const-fold up here...

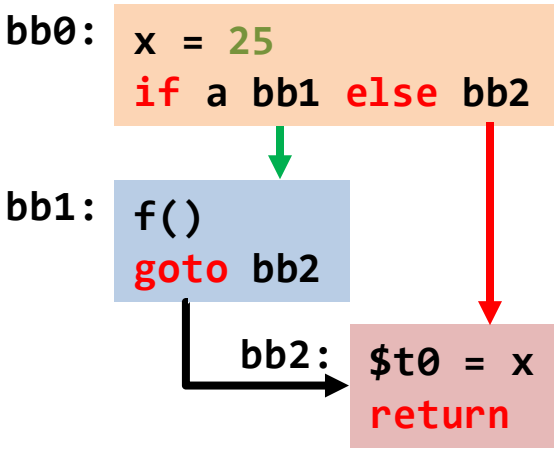
but now the assignment to **$t0** is in a different BB!

if we want our optimizations to be worthwhile at all, we're going to have to make them **operate on the whole CFG.**

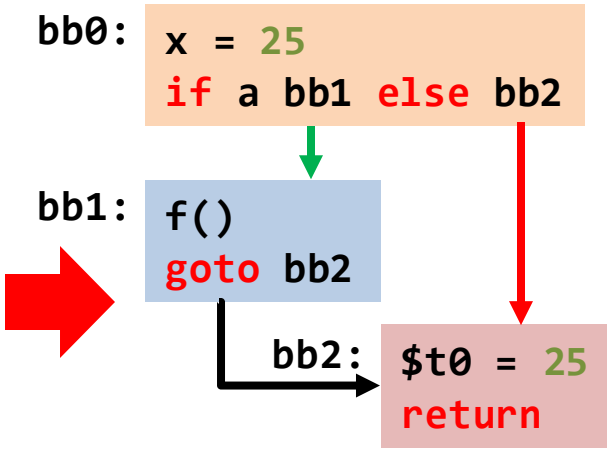
intuitively, the above *should* be possible to optimize, no?

Trying to intuit our way through it

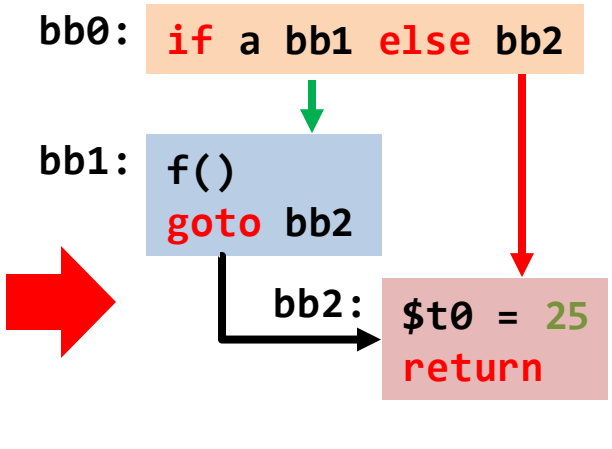
- we're going to use the same rules for copy prop/dead stores as before, but we'll look at *all* the BBs instead of just one at a time.



x is only *assigned* once, so copy its RHS everywhere it's used.



now **x** is never *used*, so delete it.

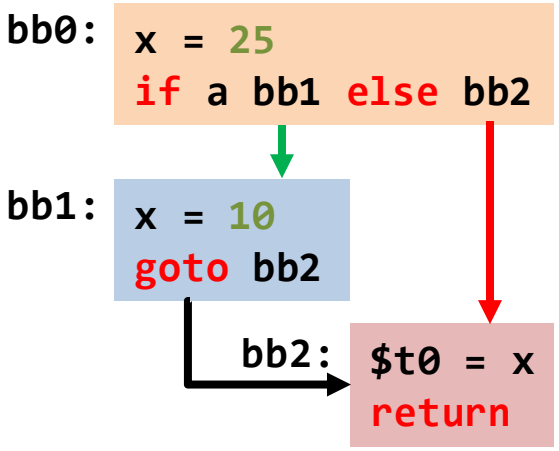


that wasn't so bad, was it?

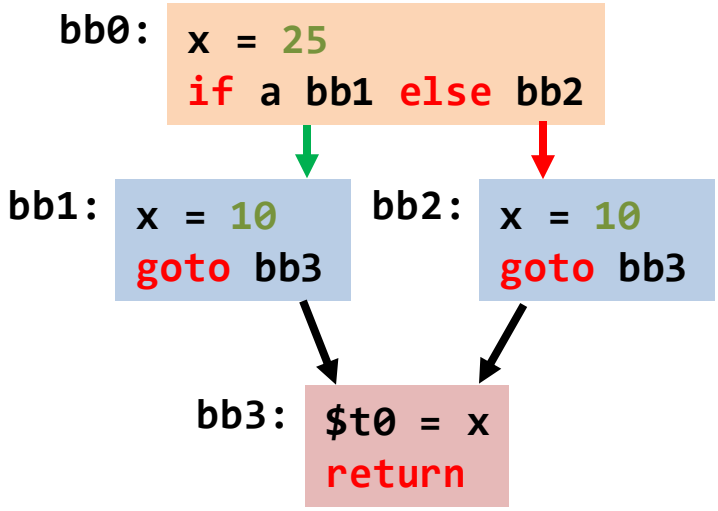
well...

Intuit THIS

- if **x** *is* reassigned, does that *always* mean we can't propagate?



here, **x** has different values on each path to **bb2**, so we *can't* propagate.

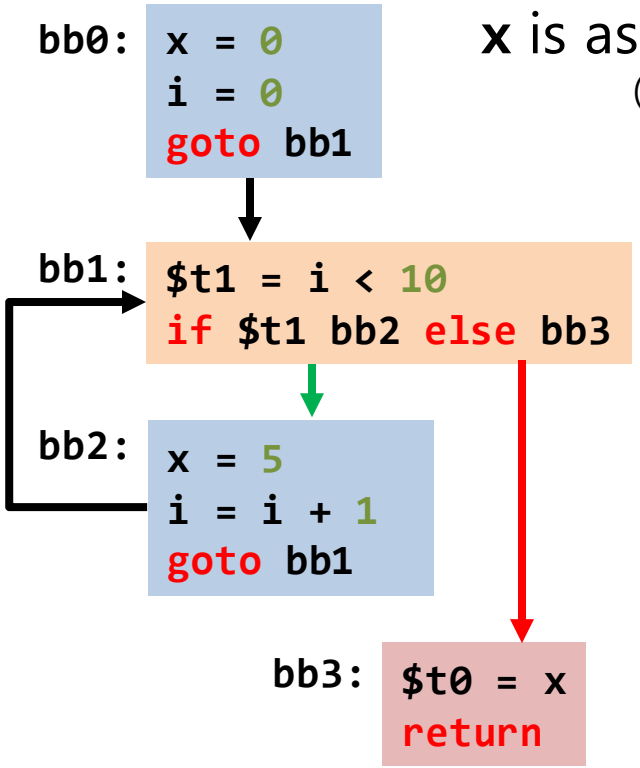


but here, although **x** is assigned in multiple places, it has *the same value* on both paths to **bb3**, so… we *can* propagate it to **$t0 = 10**???

False confidence

- it only gets worse.

```
let x = 0;
for i in 0, 10 {
  x = 5;
}
return x;
```



intuitively, we can simplify this whole function to `return 5;` but **how do you prove that?**

`x` is assigned twice, or maybe 11 times?
(do assignments in loops count as "once"?)

but the first assignment `x = 0` is never used...

and the assignment(s) in the loop are redundant...

(does that mean the whole loop can be removed?)



We need some RIGOR

- optimizations are basically **proofs.**
  - if you can **prove** that a variable is never read, you can remove it.
  - if you can **prove** that a variable only ever holds a constant value, you can replace all uses of it with that constant.
- what we need is some kind of framework to build these proofs from.
  - many optimizations have the same kind of "algorithmic shape."
  - most of them have repeated steps that stem from a common underlying reason.
- so let's talk about **data flow analysis.**

Data Flow Analysis

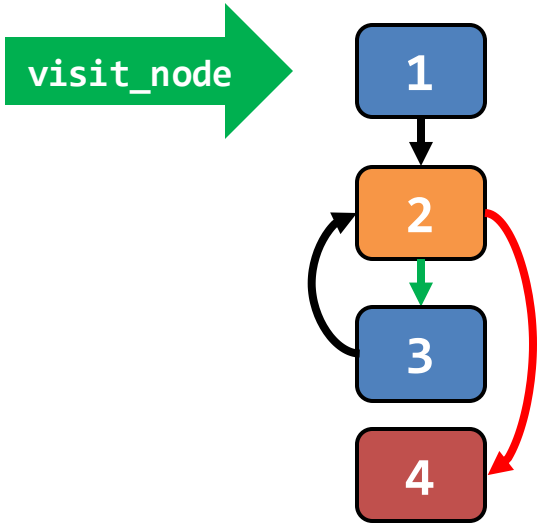
Coming for a visit (animated)

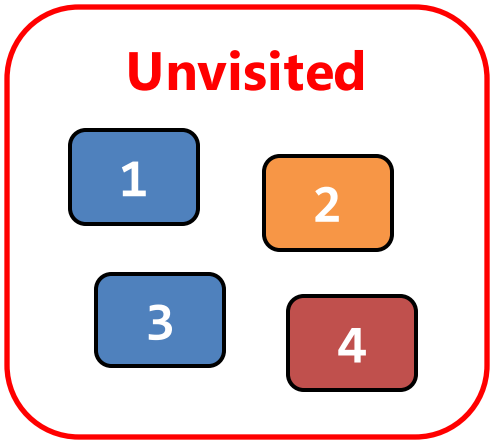
- remember this graph-visiting algorithm?

```
fn visit_node(n, visited) {
  if visited[n] { return; }
  visited[n] = true;
  for s in n.successors() {
    visit_node(s, visited);
  }
}
```

the **`visited`** set records which nodes have already been visited, and is necessary to prevent infinite loops.

so you can imagine all the nodes starting in an "unvisited" set, and gradually being moved to **`visited`**.





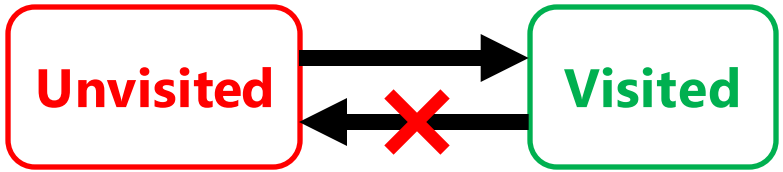


Why does it terminate?

- it might seem silly to ask, but **termination** is *crucial* to being able to specify optimizations that don't get our compiler stuck in a loop.

1. every node is in one of a **finite number** of sets. (here, it's one of two sets.)

4. on every step, we move **at least one node** from one set to another.



3. once a node reaches the "last" set, **we don't look at it anymore.**

2. nodes can move from one set to another, **but only in one direction.**

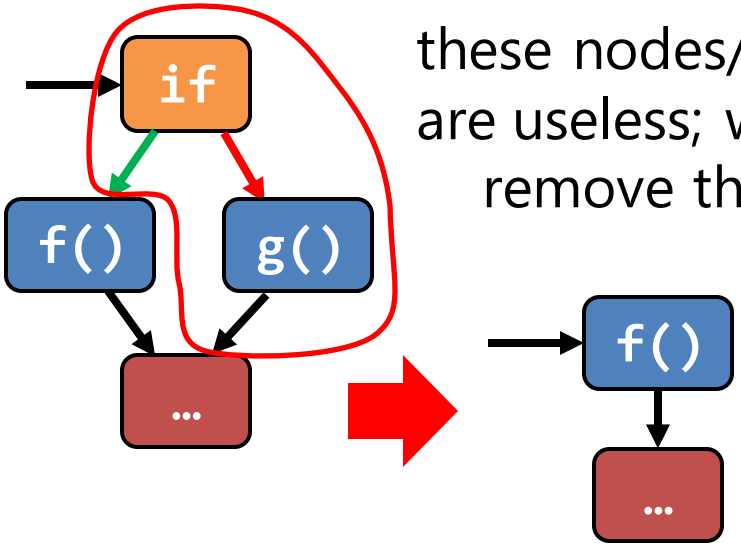
therefore, the big-O upper bound on the number of steps in the algorithm is **the number of nodes** multiplied by **the number of sets** minus one.

A simple control flow optimization

- in this function, can we ever run the **else** code (g())?

```
if true {
  f();
} else {
  g();
}
```

no. what does that look like in the CFG?



these nodes/edges are useless; we can remove them.

this can come up in real code:

```
const FEATURE_ENABLED = true;
...
if FEATURE_ENABLED {
  f();
} else {
  g();
}
```

or, the condition may have become a constant due to other optimizations.

Tweaking the visitor algorithm

- we want to detect if a BB is **unreachable,** meaning it can never run.
- we'll modify the visiting algorithm in a simple way to do this.

```
fn visit_node(n, visited) {
  if visited[n] { return; }
  visited[n] = true;
  let t = n.terminator;
  if t's condition is constant true {
    visit_node(t.true_side, visited);
  } else if it's constant false {
    visit_node(t.false_side, visited);
  } else {
    for s in n.successors() {
      visit_node(s, visited);
    }
  }
}
```

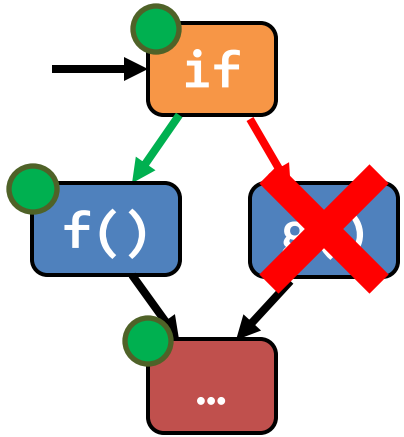
I'm paraphrasing the "real code" but the idea is simple: **if the condition is constant, only recursively visit the BB that corresponds to it.**

at the end of running this, **any nodes *not* in the visited set** are **unreachable** and can be removed from the CFG.

Running it on this CFG (animated)

- we'll mark any visited node with a green circle.

```
if true {
  f();
} else {
  g();
}
```



done. now we can see that the **`else`** node (**`g()`**) was **not visited,** and is therefore **not reachable.**

(how we remove it from the CFG is a separate issue, but it's not super complicated.)

do you remember another algorithm that worked like this? you start at a **root** and **mark** reachable things, then **sweep away** anything that isn't reachable… ;o

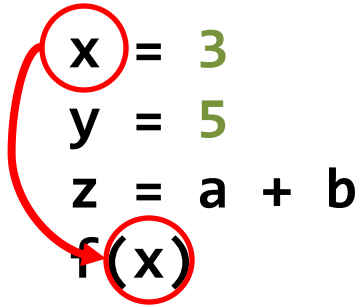
Forward analysis

- the way this (and most optimizations) works is by **transferring some kind of "knowledge"** from BB to BB by following edges.
  - here, that knowledge is the reachability of a node.
  - when we enter **`visit_node`**, we know that **n** is reachable…
  - …and that reachability is **transferred** to its successors.
- a **forward** analysis spreads this knowledge forward *with* the edges: *from* the **predecessors** *to* the **successors.**
  - a **backward** analysis does the opposite – from successors to predecessors – but we won't see one of those until next time.
- so: let's use this new knowledge to solve the problems we encountered before!

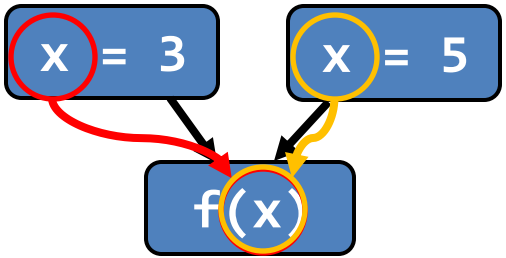
Global Constant Copy Propagation (GCCP)

Defs and Uses

- a **def** of a variable is when you assign it (it appears on ='s LHS).
  - defs of **x**: `x = 0`, `x = y`
- a **use** of a variable is when you get its value.
  - uses of **x**: `z = x`, `if x`, `f(x)`
- we'll use **def-use** to mean a pair of def and use, where the def sets the value that the use gets.



here's a def-use.



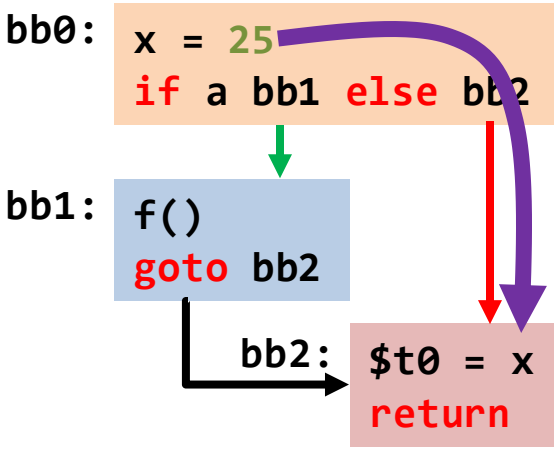
there can be multiple defs for one use, too.

What is Global Constant Copy Propagation?

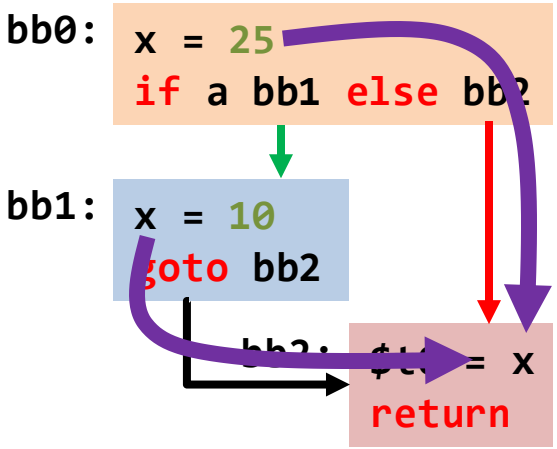
- remember that **copy propagation** said that if we have an instruction of the form "**x = y**", and **x** is never reassigned, then we can replace all uses of **x** with **y**.
- **constant copy propagation** is the same thing, but only in the cases where the RHS of "**x = y**" is a constant.
  - and **global constant copy propagation** is that, but applied to the entire CFG instead of just one BB!
- for the purposes of this example, we will *not* be using SSA, so variables **can be reassigned.**
  - essentially SSA makes it so every use has exactly one def, which simplifies some things…
  - but the details of SSA are too much for this course, so we'll stick with non-SSA for these global optimizations.

Global Constant Copy Propagation

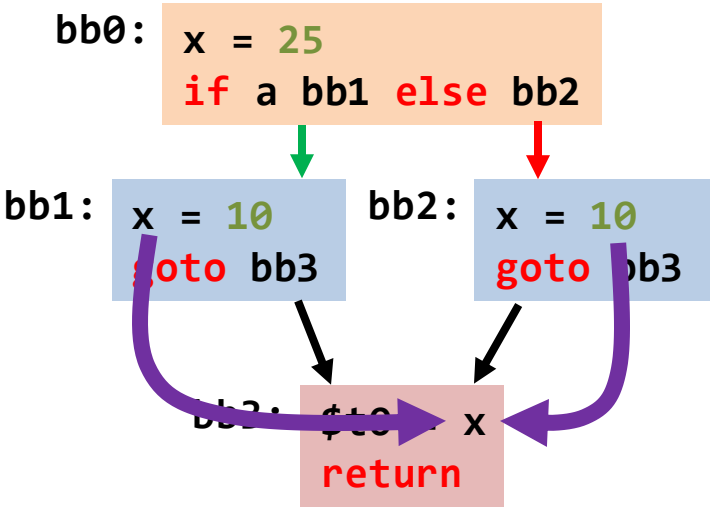
- *if* we are looking at a use of **x…**
- *and,* **every** def for that use of **x** set it to some constant **C**…
- *then,* we can replace that use of **x** with that constant **C**.



1 def-use, sets x to 25; we **can** replace x with 25.



2 def-uses; set x to **different values;** no good.



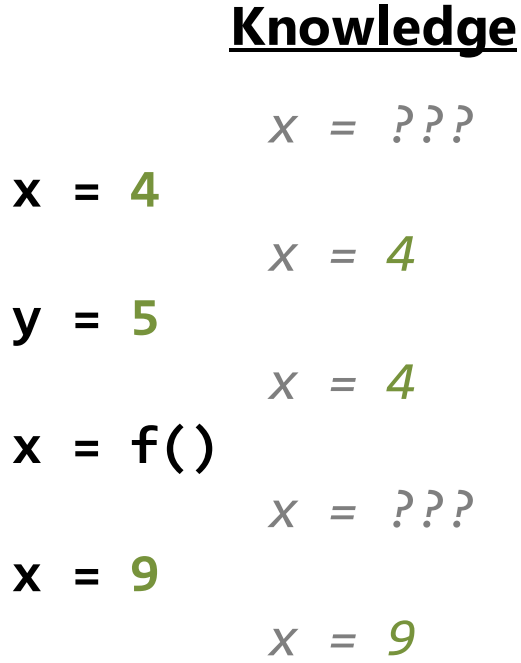
2 def-uses; both set x to **same value;** we **can** replace x with 10.

From the bottom up (animated)

- that's cool and all, but how do we start implementing this?
- let's start by looking at the **instructions** within a basic block. we'll focus on the variable **x** right now.

before this code runs, we don't know what's in **x**.

but each line **changes what we know.**



the **change in knowledge** from one step to the next is formally known as the **transfer function.**

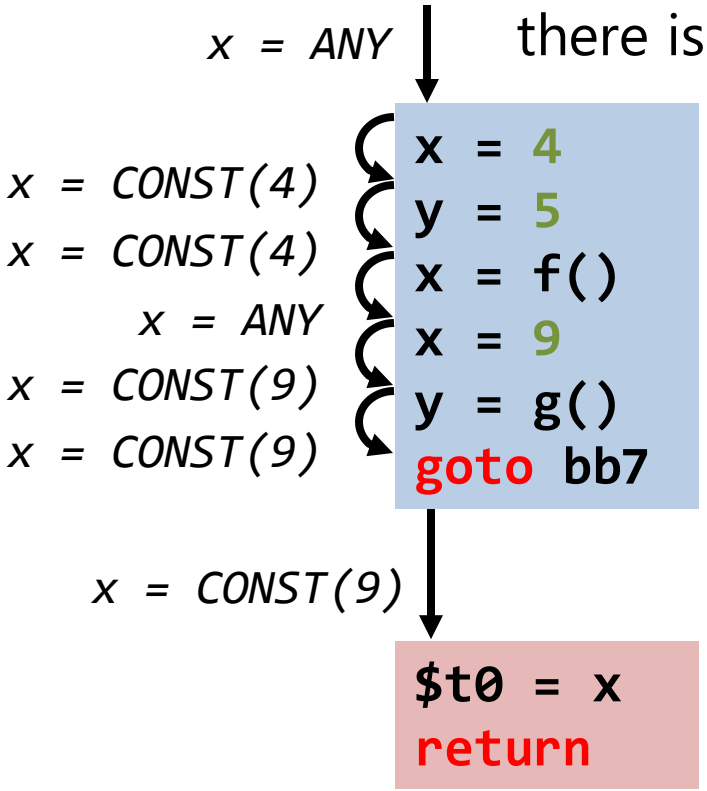
so let's be a bit more rigorous about this.

Our states and transfer function

- the "knowledge" is properly called the **state,** and we have two states:
  - **`x = ANY`**, meaning that **x** could be one of several values; and
  - **`x = CONST(c)`**, meaning **x** holds a constant **c**. (e.g. **`CONST(7)`**)
- each instruction has an ***in*-state** and an ***out*-state**...
  - and the out-state of one instruction is the in-state of the next.
- the **transfer function** takes an instruction and its in-state, and produces its out-state.
- the transfer function here is simple:
  - if the instruction is of the form **`x = c`** for some constant **c**,
    - then the out-state is **`CONST(c)`**.
  - else, if the instruction is of the form **`x = ...`** for *any other* RHS,
    - then the out-state is **`ANY`**.
  - else, the out-state is **the in-state,** unmodified.

Lather, rinse, repeat

- we can use this transfer function to compute the out-state for **a basic block as a whole,** as well.



there is some in-state for this BB.

we repeatedly apply the transfer function to the instructions inside…

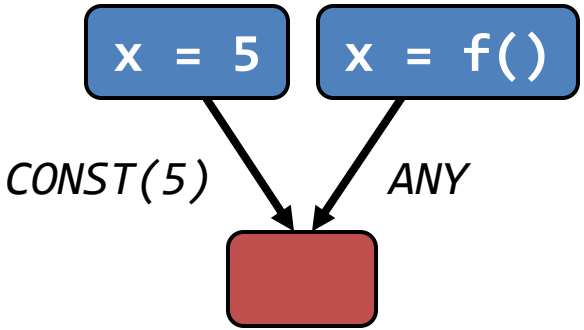
and that gives us the BB's out-state.

and as you might imagine, that can become the in-state for the **next** BB!

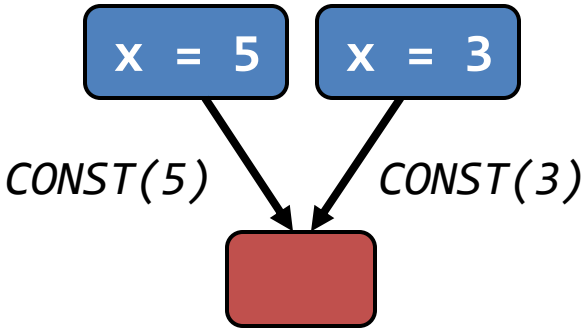
…but wait, BBs can have **multiple predecessors.**

The join function

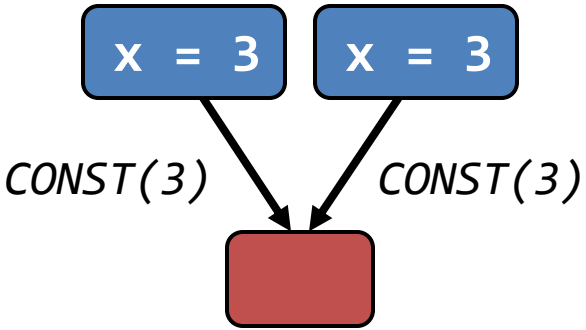
- a BB's predecessors may all be feeding **different states** into it.
- the **join function** combines those states to produce a BB's in-state.
- let's look at some examples. the labels are the state for **x**.



so, red's in-state must be **x = ANY**.



still, red's in-state must be **x = ANY**.

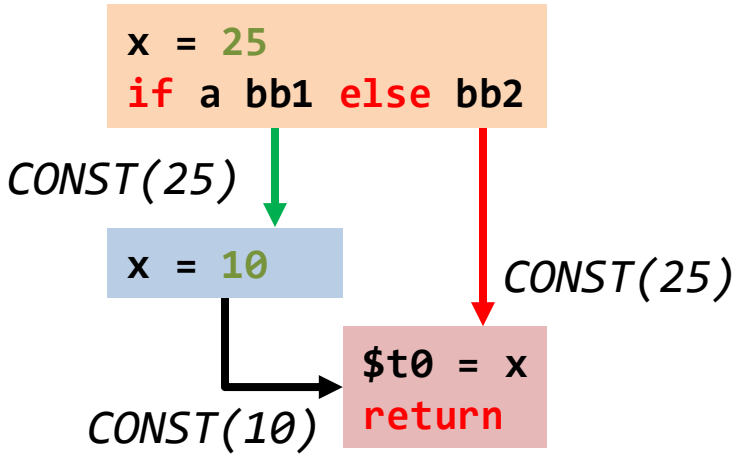


success! red's in-state is **x = CONST(3)**.

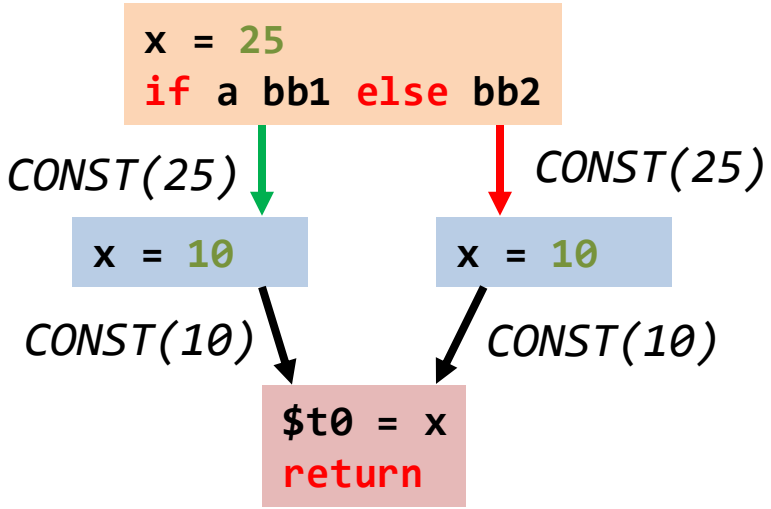
so: if **all** the predecessors say **CONST(c)** for the *same* constant **c**, then the in-state is **CONST(c)**; otherwise, it's **ANY**.

Let's try it out!

- here are two functions from before. let's **annotate the edges with the state for x.**



so, red's in-state must be **x = ANY**.



so, red's in-state is **x = CONST(10)**!

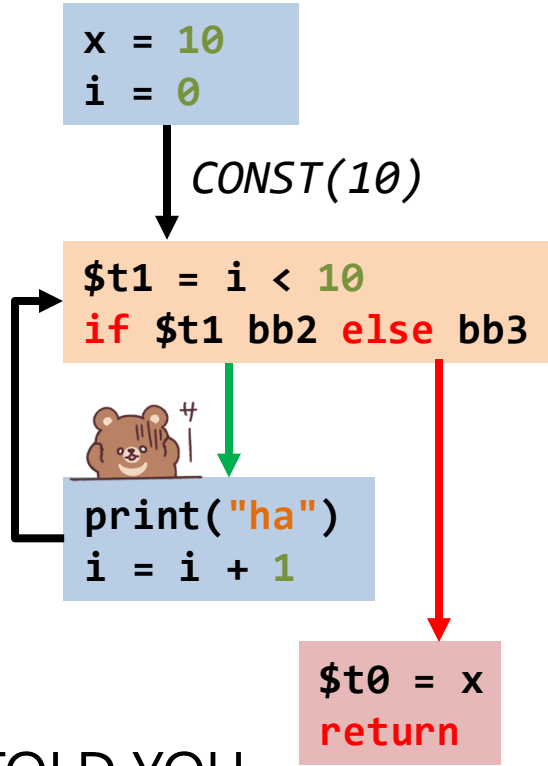
problem solved!
.....right?

The catch

- let's try it on this CFG! again, we're annotating the **state for x.**

wait, there's nothing on this edge. **then how do we compute the in-state for the orange BB?**

and if we try to follow it backwards, we end up back at the same orange BB!



IT KEEPS HAPPENING! I TOLD YOU ABOUT CYCLIC GRAPHS BRO!!

Fixing it

Oh yeah, we forgot a state

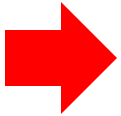
- we need **one more state** to indicate that we haven't visited that instruction/BB, **`UNK`** for "unknown."
- it will be the initial value: **every** instruction/BB's in-state and out-state will be set to **`UNK`** before the algorithm begins.
- our transfer function isn't going to have to change, fortunately.
  - it implicitly handles **`UNK`** in the "else" case: unknown in, unknown out!
- but there's the other function...

The new, improved join function

- here's where things get a bit weird, but it will all work out.

if **any** predecessor says **x = ANY**, then the output is **x = ANY**.

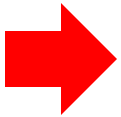
```
p1: x = CONST(10)
p2: x = CONST(4)
p3: x = ANY
p4: x = UNK
```



```
x = ANY
```

if **all** predecessors say **x = UNK**, then the output is **x = UNK**.

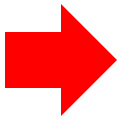
```
p1: x = UNK
p2: x = UNK
p3: x = UNK
p4: x = UNK
```



```
x = UNK
```

if the predecessors are **a mix** of **x = UNK** and **x = CONST**, then ignore the **UNK**s, and the output is **CONST** or **ANY** like before.

```
p1: x = UNK
p2: x = CONST(3)
p3: x = UNK
p4: x = CONST(3)
```

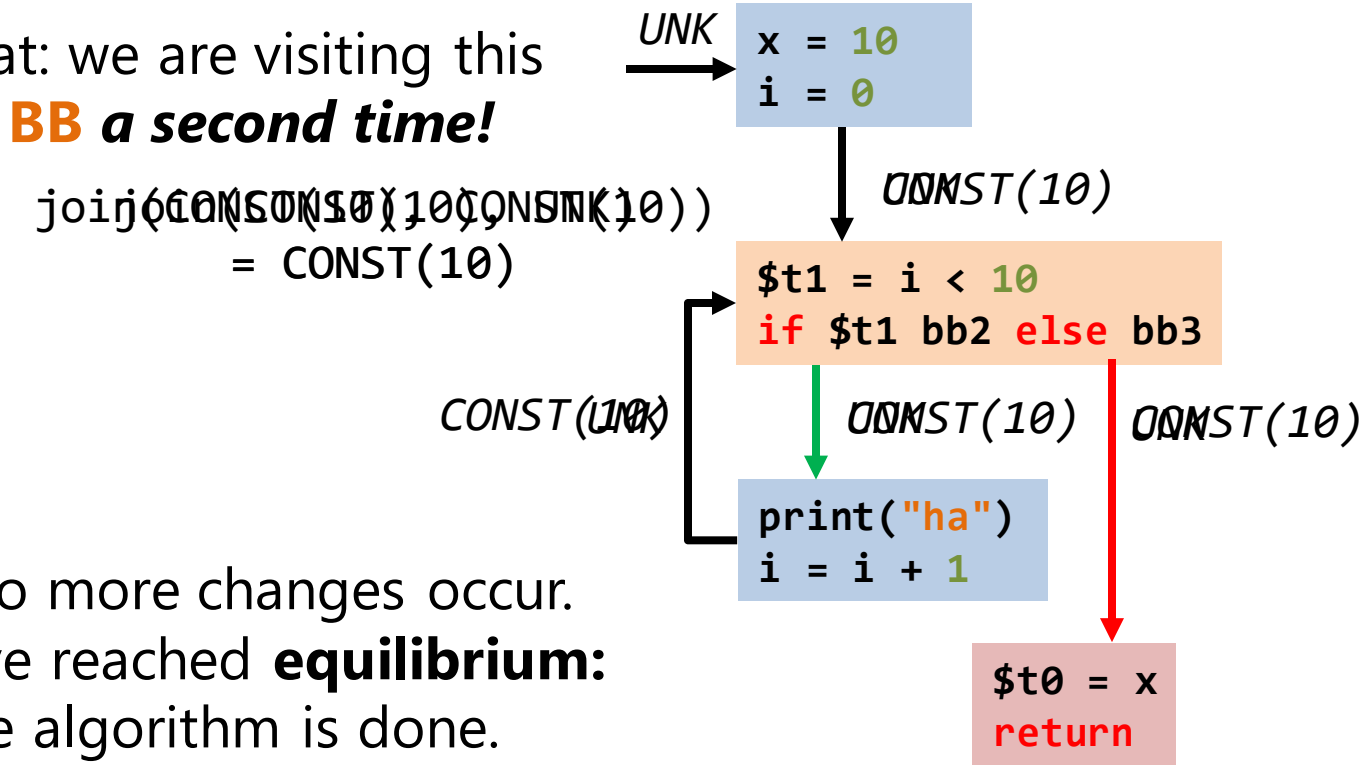


```
x = CONST(3)
```

Let's watch it go (animated, important, you have to watch this)

- if these rules seem strange, wait till you see how it behaves.
- notice that x starts off as **UNK** everywhere.

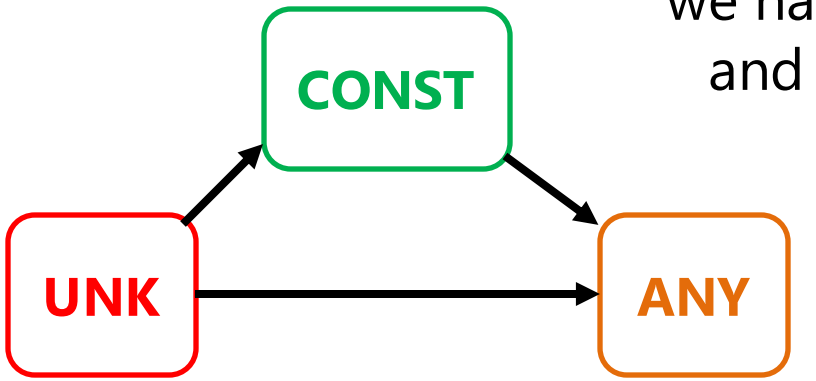
guess what: we are visiting this **orange BB *a second time!***



but no more changes occur.
we have reached **equilibrium:**
the algorithm is done.

Sometimes one visit is not enough

- because each visit to a BB might move it from one set (**UNK**) to another (say, **CONST(10)**)... and because we have **three sets...**
  - then we may have to visit a BB **more than one time!**
- we're not risking an infinite loop though. why?

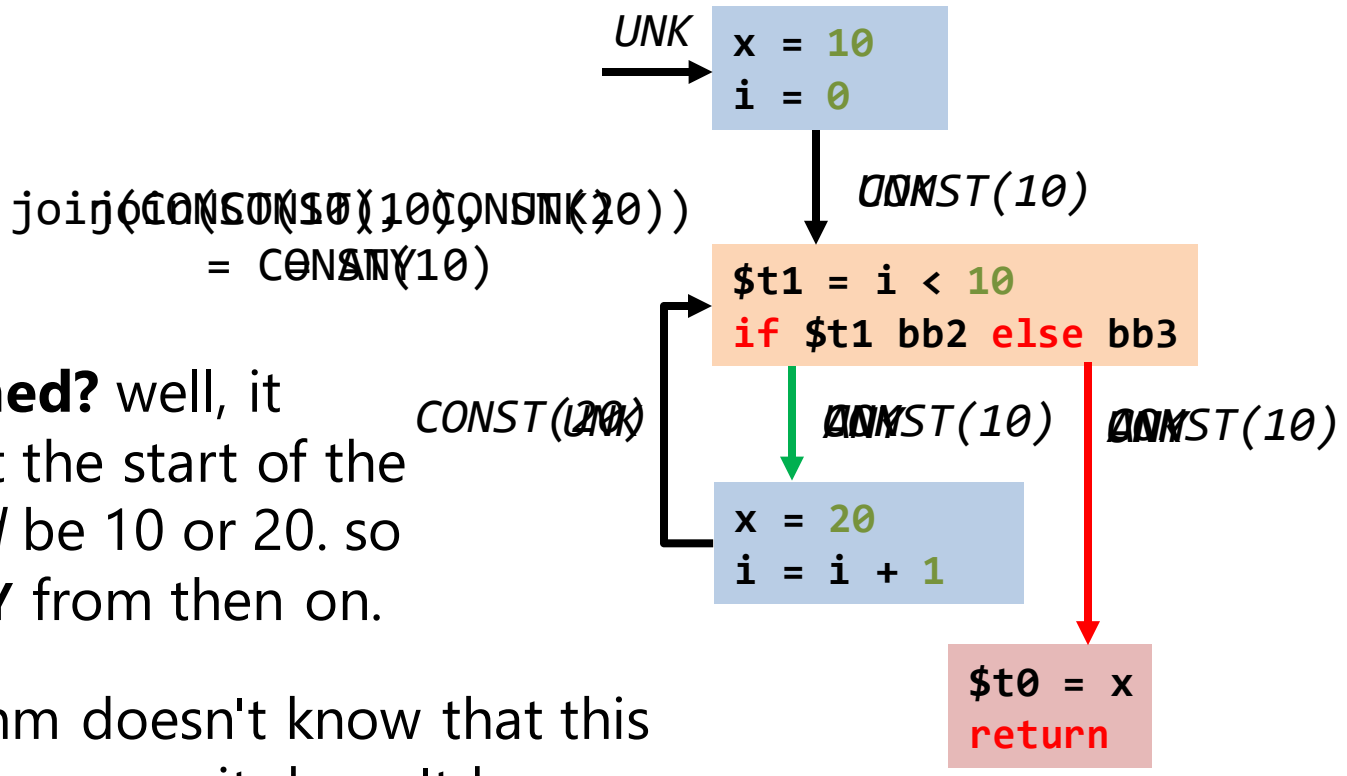


we have a finite number of states (3), and the transitions between them are **unidirectional.**

that's all we need to prove to guarantee termination. nice.

Shortcomings (animated)

- now let's see what happens when we reassign **x** in the loop.



what happened? well, it determined that at the start of the orange BB, **x** *could* be 10 or 20. so it assumes it's **ANY** from then on.

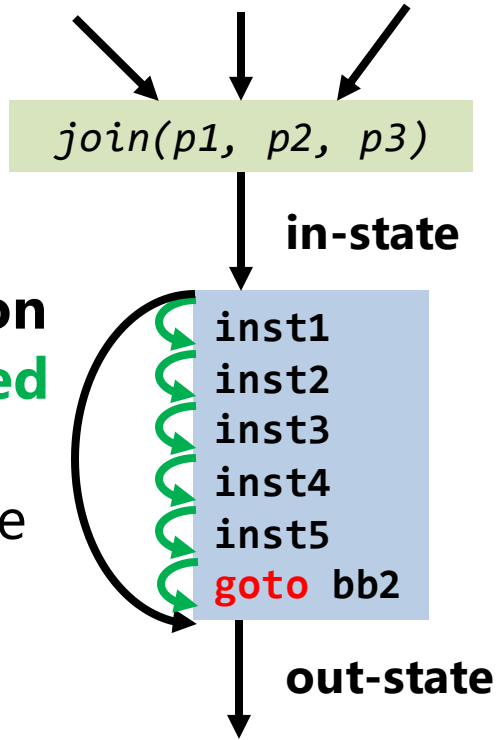
the algorithm doesn't know that this loop always runs. it doesn't know anything about loops at all!

It never hurts to *not* optimize

- the example on the previous slide is an example of the algorithm being a little cautious.
  - yeah, *we* can see that **`x = 20`** at the **`return`**, but **that wouldn't be true if the loop never ran!**
  - consider a slight modification where the loop upper bound is an argument – in that case, the loop may run 0 times.
- what you *don't* want is for your algorithm to **optimize in a situation where it shouldn't.**
  - cause that's a broken proof, and you'll get a broken program.
- sometimes an optimization pass won't find anything to do.
  - that's okay. there's no judgment. it can't know until it tries.
- and if it doesn't optimize anything, you'll still have a correct program.

Summing it up

- to recap how dataflow analysis works:



state flows **from a BB's predecessors** into its join function, which computes the **in-state** for the BB.

the **transfer function** is **repeatedly applied** to its instructions, which computes the BB's **out-state.**

this is repeated for **every BB** until the in- and out-states reach **equilibrium:** they stop changing.

as long as there are a **finite number of states,** and they change **monotonically,** this algorithm will terminate.

Liveness

time check ≤ 87

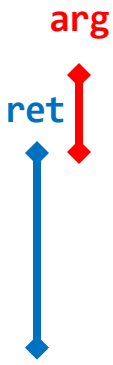
Liveness

- a local variable is **live** if its value will be **use**d in the future.
  - this is *not* its **lifetime;** liveness can be – and often is – **shorter!**
- it lasts from a **def** until the **last use** of that def.

arguments are "**def**'ed" at the start of the function.

arg is only used once on the first line, so it's dead after that.

ret becomes live when we declare it, and lives until its **last** use in the **return** statement.




```
fn lifey(arg: int): int {
  let ret = arg + 10;
  println_i(ret);
  return ret;
}
```

these lines are the locals' **liveness ranges.**

One local, many ranges

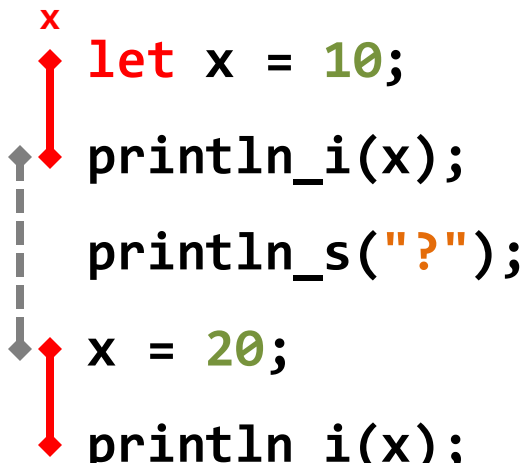
- sometimes, a local can be live and dead **multiple times!**

x is live *twice* in this code...

with a sort of "dead zone" in between.

this seems weird, but it's telling us something useful: **x is behaving like *two different variables* in this code.**

(remember SSA? this feels like a step towards it...)



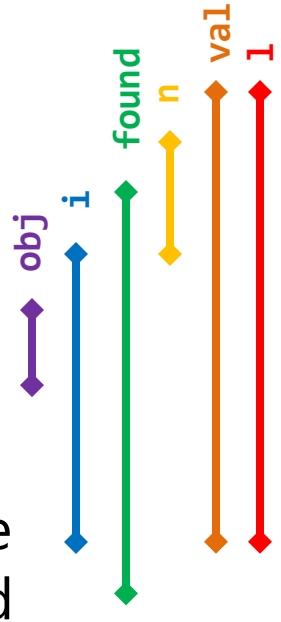
liveness is the basis for a lot of other optimizations (and error checking!), and we'll see other examples along the way.

Two perspectives

- we already saw that we can view liveness as a **set of ranges** of instructions (and basic blocks) during which a variable is live.
- but another view becomes useful when you have **multiple variables.**

now we can answer: at any point in this function, ***which* variables are live?**

this is *very* relevant to register allocation, since we're trying to map these variables onto the limited CPU registers.




```
fn has(l: List, val: int): bool {
  let n = l.length();
  let found = false;
  for i in 0, n {
    let obj = l.get(i);
    if obj.value() == val {
      found = true;
    }
  }
  return found;
}
```

Trying (and failing) to determine liveness

- below, the O and X say whether x is alive O or dead X.
  - the state is tracked *between* instructions, hence the misalignment.

let's assume by default that it's dead.

this def seems to make **x** live.

here's a use of **x**. but **is x used again after this?**

let's *assume* it'll be used again...

another use. let's keep assuming.

uh oh. it's the end. clearly, **x**'s last use was in **x == y**. so this is wrong...



```
x = 5
println_i(x)
y = 10
println_i(y)
z = x == y
println_b(z)
```